汇编命令速查表：含义与读题要点分栏

ICS 测后补强 · 发布版 · 2026–10–13 · 知识讲解与三栏命令表

保留旧版89条的阅读顺序与编号，新增具体写法放在末尾，共95条。旧表中的“第1至5讲”指本套复习讲次；“往年题精选”单独分组。不是全部x86指令清单，也不把每条说成必考。

适用x86–64/Linux/System V、AT&T语法。S是源，D是目的；R64为64位通用寄存器。S8/D8等标明位宽。通常写“源,目的”；b/w/l/q表示8/16/32/64位。cc是条件占位符，不能把setcc/jcc中的条件字母硬读成宽度。

读每条时问四句：从哪里读？做什么？写到哪里？改不改标志？ 表中“读写次数”仅指题目采用的体系结构教学口径；机器溢出、移位规则与C规则分开判断。

返回第6讲详细解释

第1讲首次出现：数据、运算与扩展

命令	含义	读题要点
`01 movl S, D`	把源的32位复制到目的。	写eax等32位寄存器会清对应64位寄存器高32位；只读eax不会清零。
`02 mov（指令族）`	把一个地方的值复制到另一个地方；源值保留。	AT&T通常按“源,目的”写；普通mov不能直接从一处内存复制到另一处内存。
`03 addb S, D`	把源与目的原值相加，把低8位写回目的。	目的既要读又要写；会改条件码。C的char加法通常先提升为int，不能直接照搬addb。
`04 add（指令族）`	目的原值加上源值，结果放回目的。	宽度看b/w/l/q；会改条件码。定宽机器截断不等于C有符号溢出规则。
`05 shl / sal k, D`	把位向左移，右边补0，左边移出的位丢弃；sal与shl效果相同。	q操作64位，计数取低6位；l操作32位，取低5位。可用立即数或cl作计数，不能直接用rbx。
`06 shr k, D`	把位向右移，左边补0。	常用于无符号数。不要与左侧复制符号位的sar混淆。
`07 sar k, D`	把位向右移，左边补原最高位的值。	负数相当于除以2的幂后向下取整；C整数除法向零截断，负数时可能差1。
`08 movb S, D`	复制1字节，即8位。	写al只改rax最低字节，其他位保留；不会自动得到完整的64位0或1。
`09 movw S, D`	复制2字节，即16位。	写ax只改rax低16位，其他位保留。
`10 movq S, D`	复制8字节，即64位。	rdi是寄存器内容；(%rdi)在mov中是该地址处的内存。方向决定读内存还是写内存。
`11 subq S, D`	从目的原值中减去源值，写回64位结果。	第二项减第一项；减rsp是在预留栈空间，不是在初始化栈内存。会改标志。
`12 addq S, D`	把源值加到目的原值，写回64位结果。	加rsp通常回收栈空间；原内存字节没有被擦除。会改标志。
`13 imulq S, D`	双操作数形式：把源与目的相乘，将低64位乘积写回目的。	目的也提供一个乘数；这不是单操作数的rdx:rax完整乘积形式。

命令	含义	读题要点
14 andq S, D	逐位做“与”，结果放回目的；掩码1保留原位，0清零。	不是C的逻辑&&；会改标志，CF与OF清零。
15 orq S, D	逐位做“或”，结果放回目的；掩码1强制置1，0不改变原位。	不是C的逻辑或；会改标志，CF与OF清零。
16 xorq S, D	逐位做“异或”：相同得0，不同得1；结果放回目的。	与自己异或能清零；也会覆盖比较标志。掩码1翻转、0不变。
17 lea / leaq 地址式, D	算出括号里那套地址算式，把算出的数放进寄存器。	不读取该地址处的内存，不改标志。也可用来算5×x等算术；括号不一定代表访存。
18 movz（零扩展族）	把窄值复制到宽寄存器，新增高位全部补0。	z表示zero；后两个字母是源宽度、目标宽度。用于保留无符号数值。
19 movs（符号扩展族）	把窄值复制到宽寄存器，新增高位都复制源最高位。	s表示sign；用于保留补码有符号数值。此处不是字符串传送movs的用法。
20 movzbl S8, D32	取8位源，左边补0，形成32位结果。	常跟在setcc后整理成int的0/1；写eax还会清rax高32位。
21 movsbq S8, D64	取8位源，按源符号位扩展成64位。	源FD扩展为FFFFFFFFFFFFFFFD，64位读为−3；正数则补0。
22 movslq S32, D64	取32位源，按源符号位扩展成64位。	若eax最高位为1，新rax高32位全1；与movl写32位目标造成高位清零不同。
23 movsbl S8, D32	取8位源，按源符号位扩展成32位。	先扩到32位，若写eax又清rax高32位；不等于8→64的movsbq。
24 addl S, D	把源与目的的低32位相加，写回32位结果。	写eax等会清相应64位寄存器高32位；标志描述32位运算。

第2讲首次出现：比较、条件与标志

命令	含义	读题要点
25 cmpq S, D	用64位“目的−源”的结果设置条件码，两个操作数不被改写。	cmp b,a判断a与b；本身不区分有符号、无符号，后续条件指令决定读法。
26 testq S, D	用64位“目的按位与源”的结果设置标志，不写回该结果。	testq rax,rax常检查是否为0；CF、OF清零，不等于比较两个数谁大。
27 testb S, D	只按8位做与测试并设置标志，操作数不变。	testb al,al仅检查这个字节；不检查eax或rax的其他位。
28 cmp（指令族）	做一次减法式比较，只留下条件码。	读条件跳转前，先确定减法顺序和宽度；后面其他算术可能覆盖这些标志。
29 setcc D8	条件成立写一个字节1，否则写一个字节0。	cc是条件占位符；所有setcc都写8位，不清目的寄存器其他部分，也不改标志。
30 jcc 标签	条件成立转去标签，否则执行紧随其后的指令。	读取已经存在的标志；不重新比较，不自动保存返回地址。
31 cmovcc S, D	条件成立，将源复制到目的寄存器；不成立保留该宽度的默认值。	先准备候选和标志；无8位或立即数源形式。内存源不保证条件假就不读；32位形式仍清目标高32位。

命令	含义	读题要点
32 sete D8	相等条件成立时写字节1，否则写0。	看ZF=1；若要完整int布尔值，通常还需movzbl或预先正确清零。
33 je 标签	ZF=1时跳转。	常用于相等或结果为零；jz是同义名字。条件含义取决于谁设置了ZF。
34 cmove S, D	ZF=1时用源值替换目的候选。	不成立保留目标宽度内旧值；目的须为寄存器。32位形式还要考虑高32位清零。
35 sub（指令族）	从目的原值中减去源，写回结果。	会改标志；目的一源，别颠倒。立即数放源端时带$。
36 inc D	把目的加1。	改ZF、SF、OF等但不改CF；不能一概当成对标志完全等同于add $1。
37 dec D	把目的减1。	同样保留CF；若后续判断借位，不能把旧CF误当成这次减1新产生的。
38 sete / setz; je / jz; cmove / cmovz	相等/结果为零：set写0或1，j决定跳不跳，cmov决定换不换值。	条件ZF=1。三者动作不同，但检测同一个条件；以cmp b,a后解释a与b。
39 setne / setnz; jne / jnz; cmovne / cmovnz	不相等/结果非零：分别写布尔值、跳转、换候选。	条件ZF=0。ne和nz同义；不要求检查CF或SF。
40 setl; jl; cmovl	按有符号数判断“小于”。	条件SF≠OF；l是less，不是32位long。只看SF会漏掉溢出修正。
41 setle; jle; cmovle	按有符号数判断“小于等于”。	条件SF≠OF或ZF=1；比小于多了相等的情形。
42 setg; jg; cmovg	按有符号数判断“大于”。	条件SF=OF且ZF=0；既不能小于，也不能相等。
43 setge; jge; cmovge	按有符号数判断“大于等于”。	条件SF=OF；允许ZF=1，不必另要求非零。
44 setb; jb; cmovb	按无符号数判断“小于”。	条件CF=1；减法有借位。b是below，不是byte宽度后缀。
45 setbe; jbe; cmovbe	按无符号数判断“小于等于”。	条件CF=1或ZF=1；不要与有符号le混用。
46 seta; ja; cmova	按无符号数判断“大于”。	条件CF=0且ZF=0；a是above。负有符号位模式按无符号读可能很大。

命令	含义	读题要点
47 setae; jae; cmovae	按无符号数判断“大于等于”。	条件CF=0；与是否有符号溢出OF无关。
48 cmpl S, D	按32位“目的−源”设置标志。	这里l是32位宽度；不改目的，也就没有“写eax清高位”。
49 cmpb S, D	按8位“目的−源”设置标志。	SF看8位结果最高位；CF看无符号借位；OF看有符号溢出。
50 ret	从当前栈顶读返回地址，rsp加8，然后到该地址继续执行。	整数返回值早已准备在rax；ret弹的是代码地址，不是返回值，也不恢复全部寄存器。
51 xor（指令族）	逐位异或，结果写回目的。	同寄存器异或为0；会改标志，不能随便插在cmp与jcc/cmovcc之间。
52 subb S, D	按8位做“目的−源”，只保留低8位。	CF与OF含义不同；8位结果最高位为1不必然说明原数学差为负。
53 testl S, D	用低32位做与测试，只写标志，不改操作数。	testl eax,eax检查低32位是否全0；不会清rax高32位。

第3讲首次出现：跳转表、除法与调用

命令	含义	读题要点
54 jmp 标签 / jmp *操作数	无条件转到另一个代码位置。	直接跳标签；间接跳转有*。jmp *%rax取寄存器目标，jmp *内存式要读取目标地址；不压返回地址。
55 .quad 地址或数值	让汇编器在这里放一个8字节的数据或地址。	属于汇编器指示，不是CPU命令。跳表里存目标地址，不能把它当成计算结果。
56 .section .rodata	让后续内容进入只读数据节。	属于汇编器指示；不是运行时跳转，也不表示CPU在此读一次内存。
57 .align 8	让后续内容从8字节边界开始，必要时插入填充。	这里按GNU x86语境解释；不是运行时把寄存器乘8。
58 cqto	把rax的64位有符号数扩展成rdx:rax的128位数。	rax不变；rdx全0或全1，旧rdx被覆盖。常为idivq准备被除数。
59 idivq 除数	用rdx:rax组成的128位有符号数除以给定除数。	商进rax，余数进rdx；它们执行前后角色不同。商向零截断；除零或商放不下会异常。
60 call f	压入下一条指令地址，再执行函数f。	压的是回来的位置，不是f入口地址；会改变rsp并写栈，不自动保存其他通用寄存器。
61 pop / popq D	读当前栈顶8字节到普通目标，再把rsp加8。	弹出不擦除内存；若目标是rsp等特殊形式，不能机械套普通目标步骤。
62 push / pushq S	先把rsp减8，再把源值写入新栈顶。	pushq rbx保存rbx的数值，不读取rbx指向的内存；源为rsp有特殊取值次序。
63 leave	撤销使用rbp作帧指针的常见栈帧。	常见64位布局下相当于mov rbp,rsp再pop rbp；还要ret才返回，不能用于任意栈布局。

第4讲首次出现：标量浮点

命令	含义	读题要点
64 movss S, D	传送一个32位float的位模式。	在XMM/内存之间看源目的判断读写；不把int自动转换成float。
65 movsd S, D	传送一个64位double的位模式。	这里指XMM标量传送，不是同名字符串指令；不是float→double数值转换。
66 addss S, D	把两个标量float相加，结果写回目的。	ss是单精度标量；有舍入。不会像整数add那样设置ZF/SF/CF/OF。
67 addsd S, D	把两个标量double相加，结果写回目的。	sd是双精度标量；有舍入。目的的标量旧值也是加数。
68 cvtss2sd S, D	把float数值转换成double。	有限float值能精确保留；之前已经丢掉的精度不会恢复。
69 cvtsi2sd（整数转double）	把有符号整数数值转换成double。	si代表signed integer，不代表rsi；源32/64位看操作数或完整后缀，64位整数可能舍入。
70 ucomiss S, D	比较两个float，设置ZF、PF、CF。	NaN会产生无序，此时三者均为1；不要直接套整数jl来判断浮点大小。
71 ucomisd S, D	比较两个double，设置ZF、PF、CF。	OF、SF清零；若可能有NaN，需检查PF区分无序和相等等情况。
72 cvtsi2sdl S32, D	把32位有符号整数转换成double。	末尾l指整数源32位；不是条件less，也不是目标double只占32位。
73 jp 标签	PF=1时跳转。	紧跟ucomis时常检查NaN无序；跟在其他指令后，PF含义要按该指令判断。

第5讲首次出现：具体清零写法

命令	含义	读题要点
74 xorl S, D	做32位异或，把结果写回目的。	xorl eax,eax让整个rax为0，同时改变条件码。

往年题精选首次出现的写法

命令	含义	读题要点
75 leal 地址式, D32	计算地址算式，把结果低32位放进目的寄存器。	不读该地址的内存，不改标志；写32位目标会清对应高32位。
76 jc 标签	CF=1时跳转。	与jb检查同一标志，但叫carry提醒你考虑前条指令是加法进位还是减法借位。
77 incw D16	把低16位加1，结果写回这16位。	保留CF；不清目的寄存器其他高位。
78 shlw k, D16	按16位左移，右边补0。	等价salw；传统移位计数用低5位，不是低4位。多位移位的OF不能随便认定为0。
79 subw S, D	按16位做目的−源，写回低16位。	改标志；写16位目标不清其余高位。
80 xorw S, D	按16位异或并写回。	只改目的低16位；会改条件码。

命令	含义	读题要点
81 testw S, D	按16位做与测试，结果不写回。	CF、OF清零；其他相关标志依16位测试结果设置。
82 andw S, D	按16位做与，写回目的。	和testw的区别在于确实改写目的值；可用于掩码清位。
83 movabsq 立即数, R64	本表形式把完整64位立即数放入64位寄存器。	用于普通立即数编码装不下等情形；movabs还存在绝对地址形式，应看完整操作数。
84 subl S, D	按32位做目的−源，写回结果。	改标志；写eax等会清对应64位寄存器高32位。
85 shrq k, D64	按64位逻辑右移，左边补0。	计数用低6位；65等效移1只适用于机器指令，不能据此认定C移65位合法。
86 jmpq *操作数	64位间接无条件跳转。	*表示目标来自操作数；可来自寄存器或内存。不会压返回地址。
87 and（指令族）	逐位做与，结果写回目的。	与test不同，它会修改目的；CF与OF清零。
88 retq	64位普通返回，从栈中取返回地址并跳回。	题库retq与本表ret相同；别与恢复rbp的leave混为一条。
89 callq f	64位调用，压入下一条指令地址并转到f。	题库callq与本表call相同；调用前仍要自行满足栈对齐和保存规则。

本次补强新增的具体写法

命令	含义	读题要点
90 salq $k, D64 / salq %cl, D64	把64位目的左移，空出的低位补0。	q说明数据宽度64位；次数取低6位，不是只移动6位数据。salq %rbx,%rax不是合法的普通x86写法。
91 shlq $k, D64 / shlq %cl, D64	同salq，是64位左移的另一个名字。	sal和shl没有“左边补符号位”的区别；需要复制符号位的是算术右移sar。
92 sarq $k, D64 / sarq %cl, D64	把64位目的右移，高位填原符号位。	负数右移相当于向下取整；模拟C向零整数除法时，需要按条件加偏置。
93 movzwl S16, D32	把16位源补0扩展成32位。	w是源16位，l是目标32位；写32位寄存器还会清对应高32位。
94 movswq S16, D64	把16位源按符号位扩展成64位。	最高位看源的第15位；不是看扩展后第63位再决定怎么扩展。
95 cvttsd2sil S, D32	把double向零截断成32位有符号整数。	tt指截断；普通有限且截断结果在范围内时，与C转int一致。NaN或超范围时不要硬套普通转换结论。

使用提醒与来源

条件表中的set/j/cmov按同一个条件后缀排列：set把判断写成字节0或1，j决定执行路径，cmov选择数据。除特殊说明外，表中条件关系按cmp b,a后比较a与b理解。

本表由此前速查表、本套第1至5讲及往年题精选中的指令重整而来。新增部分为本次薄弱点补充。扩展传送、条件传送和ABI的详细依据见第6讲相应段落。修订日期：2026–10–11。